

Projects: [EUPILOT](#), [MAELSTROM](#), [MExMeMo](#), [NVIDIA Lab](#), [OpenGPT-X](#), [OPTIMA](#), [SCALABLE](#)

A mathematician's introduction to transformers and large language models

Jul 13, 2022 by [Carolín](#)

DOI: [10.34732/xdvblg-qsbtyx](https://doi.org/10.34732/xdvblg-qsbtyx)

 CC-BY-SA 4.0

About

This blog post is based on a presentation I held at the “[New Trends in Computational Science in Engineering and Industrial Mathematics](#)” workshop in Magdeburg on 01/07/2022. My goal is to give a brief introduction to the state of current large language models, the OpenGPT-X project, and the transformer neural network architecture for people unfamiliar with the subject.



The audience at the workshop had a mathematics background and is assumed to have a good understanding of linear algebra, but not necessarily of neural networks. Basically, the target audience is past me from before I started working on this project with the goal of understanding the math behind transformers. The questions I want to answer are:

1. [About](#)
2. [What is a language model?](#)
3. [Deep learning architectures](#)
4. [Attention please!](#)
5. [From attention to transformers](#)
6. [Recent developments in large language models](#)
7. [Takeaways and learnings](#)
8. [Sources](#)

1. Where are matrix products performed in training large language models?
2. What makes transformers well-suited for high performance computing (HPC)?

If you find any mistakes or unclear points feel free to let me know in order to improve this post.

What is a language model?

Natural language processing deals with making the human language accessible for computations.^{1 2} Having a computer understand what you say can help in many situations. Applications of NLP include intelligent speakers, chatbots, translation, text generation, summarization and much more.

A **language model** forms the back bone of these applications. A language model is just a probability distribution. Given a sequence of words $w_{1:(t-1)} = (w_1, \dots, w_{t-1})$, a language model gives the probability of all the words in your vocabulary V to follow this sequence,

$$P(w_t | w_{1:(t-1)}), \quad w_1, \dots, w_{t-1}, w_t \in V.$$

With such a language model one can generate new texts: Start with a sentence, then choose the word with the highest probability (or sample according to probabilities) and feed the new appended sequence back into the model to generate the next word. The language model can be used to assign a probability to a sentence (using the chain rule of conditional probabilities) as

$$P(w_{1:n}) = \prod_{i=1}^n P(w_i | w_{1:(i-1)}).$$

One can imagine this to be helpful in grammar corrections for example.

There are different ways to arrive at such a language model. One could think about putting all rules of grammar and the meaning of words into a computer program. However, this is extremely difficult to do. The approach that caught on in recent years and produced very impressive language models does not require encoding explicit grammar or world knowledge. Instead, neural networks are trained on huge amounts of text and learn to form proper sentences just from the data they see.

In order to understand the broader context of the transformer architecture in NLP applications, we clarify some terms related to training and application of large language models.

1. *Pre-training*: The goal of pre-training is to provide a general language model that has a good understanding of how language is used in a variety of settings.
2. *Fine-tuning*: In fine-tuning, a pre-trained model is trained further on a (comparatively) small set of task-specific data. Before the emergence of pre-trained models, neural networks were trained from scratch for each specific application (also called *downstream task*). Using a pre-trained model uses compute resources more efficiently and can avoid overfitting. Fine-tuning can involve continued training of the whole network or parts of it (*layer freezing*).

This step is also called adaptation and may also include adapting the neural network's architecture.

3. *Inference*: When the model is deployed, for example in form of a chatbot in an online shop, inference describes computing the output (the answer of the chatbot) given a user's input, using the trained model. This corresponds to a forward-pass of the neural network.

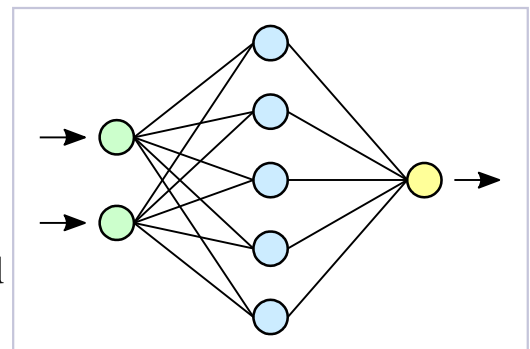
The learning methodology described by the first two steps (pre-training followed by fine-tuning) is called sequential transfer learning.³

All these steps need computing resources. The computational device of choice is typically the GPU due to the massive parallelism it provides and hardware features that make it extremely efficient in performing matrix multiplications. We will see below (in the section [Attention please!](#)) how matrix multiplications form the core of training the model. Pre-training of large models is the most computationally demanding step and happens on a supercomputer such as [JUWELS](#) at Forschungszentrum Jülich using lots (hundreds) of GPUs in parallel. Fine-tuning and inference may happen on server systems with a handful of GPUs.

Deep learning architectures

Neural networks are everywhere. You might be familiar with the basic ideas. There are many great resources to learn the foundations.^{4 5} The goal of training a neural network is to learn input-output relations from data. When a neural network is well-trained, a vector representing input data is fed to an input layer. In illustrations this is on the left (like the one to the right by [Dake & Mysid on](#)

[Wikimedia Commons](#)). Then it is processed by passing several hidden layers until it reaches an output layer. Moving from one layer to the next means multiplying the vector with a matrix, adding another vector and applying a non-linear activation function. This is called a forward-pass or forward-propagation.



The elements of the matrices are called weights, the elements of the additive vector are called biases. Weights and biases are the parameters that are learned during training. For your training data, the output given by the network should closely match the real desired output, i.e. the loss function (measure of difference between network's output and desired output) should be minimal. If this is not yet the case, we change the parameters to achieve a smaller loss. This is done using gradient descent. The gradient of the loss function with respect to the parameters is computed. The parameters are updated by adding the gradient multiplied by a step

size (called learning rate). The actual computation of the gradients uses the chain rule from calculus and involves starting at the output layer and moving backwards through the network. This is why computing the gradients is called backward propagation.

In practice, more useful heuristics are added to this process, and it works very well for many tasks. However, it is difficult to use the fully-connected neural network for NLP tasks. One problem is that the input size is fixed, and we would like to process longer as well as shorter word sequences as input. In general, a dense neural network does not represent the nature of language very well.

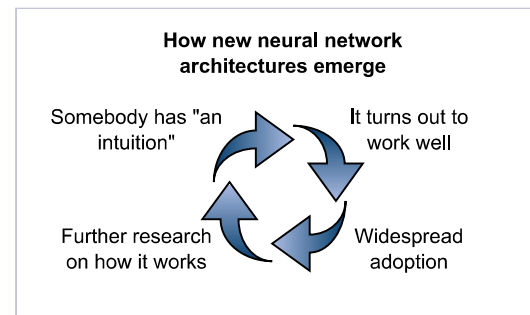
Luckily, this standard feed-forward neural network is only the most basic neural network architecture of many that were devised over the years for various applications.

In the field of NLP and language modelling, until recently, sequential models were the state of the art. These include *recurrent neural networks* (RNNs) and *long short-term memory* (LSTM) networks.⁶

RNNs apply the same neural network (with learned parameters) to every word in a sequence of words. Additionally, this neural network takes an internal state as input, which comes as output from the neural network associated to the previous word. This way the network can learn to use information from earlier words in the sequence. When one writes down the gradient of the loss function with respect to the parameters using the chain rule, one can see that the newest word has the most influence. The influence of the previous words diminishes exponentially. Intuitively, this makes sense: For choosing the next word, the most recent word is on average more important than a word further in the past. However, in practice, language is more nuanced. Some specific words in the past can be very important for choosing future words, and a smart neural network should know how to look for them. Just think of a very long relative clause for example. Older words having less influence on the gradients is therefore more of a bug than a feature, and this is called the *vanishing gradients* problem.

LSTMs alleviate this issue by introducing an extra cell state (serving as “memory”) whose exact influence is determined by gates that are defined by more learnable parameters.

One drawback remains: Both RNNs and LSTMs process their input data sequentially. Consider the forward pass: In order to apply the neural network (a series of matrix multiplications) on an input word vector x_i we also need the result



from applying the network on the previous word vector x_{i-1} . We can not stack the word vectors together in a matrix and apply a neural network all at once.

Formulating algorithms to use matrix-matrix products as main computational element is a good step forward towards the efficient use of modern compute hardware. This is true on the small scale of a single processor to the large scale of supercomputers using thousands of GPUs. Matrix-matrix products are the key.

Realizing this need, researchers started “having intuitions” about neural network architectures that employ these operations to learn to pay *attention* to other relevant words.

Attention please!

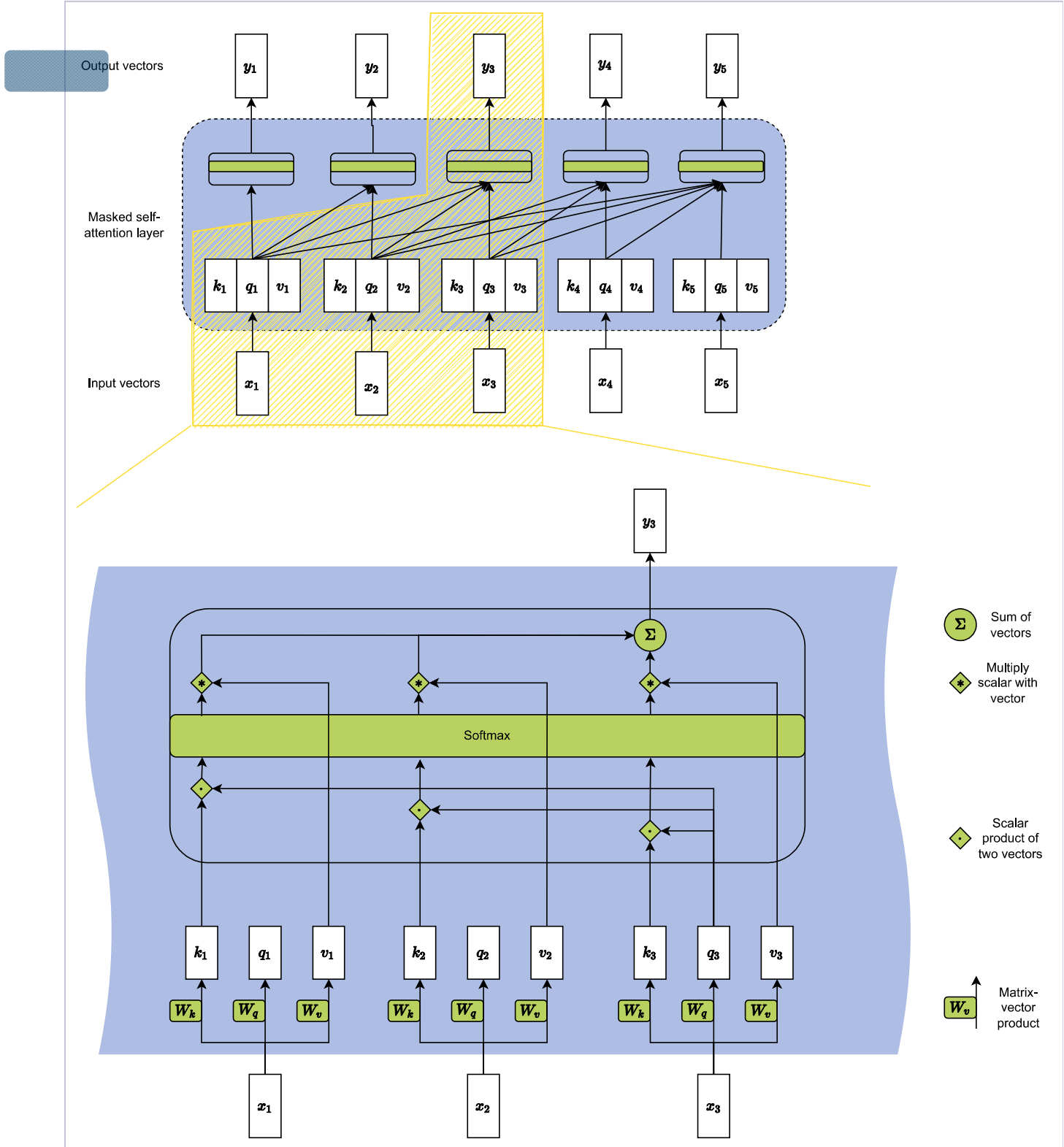
The so-called *attention* mechanism had been employed in the context of sequence models to give the model the opportunity to learn which words are relevant for the next word. The landmark paper “Attention is all you need” (2017) ⁷ showed that you do not need a recurrent network structure, and that the attention mechanism (together with some other tricks like positional encoding) is powerful enough for impressive results. The resulting neural network architecture is called a transformer.

In the following we describe a forward-pass through a (self-)attention layer, which forms the central element of a transformer block. A neural network architecture is called a transformer when it consists of several transformer blocks.

Backpropagation is taken care of by using the automatic differentiation engines of frameworks such as PyTorch or TensorFlow.

Consider a sequence of input tokens $x_1, \dots, x_n \in \mathbb{R}^{n_{\text{model}}}$ represented by vectors. Tokens are the smallest building blocks into which word sequences are divided for processing. The process of getting a sequence of tokens (represented as a series of integers referring to a vocabulary) from a text string is called tokenization. The vector representation of a token is called an embedding and spatially encodes the meaning of tokens and their relationship towards each other. In the case of transformers, word embeddings are also learned during pre-training. You can think of this as a matrix with learned entries being multiplied with a one-hot vector, i.e. choosing row i when the token is encoded as integer i . A one-hot vector is called a (standard) unit vector in numerical linear algebra.

The processing of the first three input vectors x_1, x_2, x_3 to generate an output vector y_3 is seen in the following diagram:²



Among the learned parameters of a transformer block are three matrices W_k , W_q and W_v . They transform an input vector x_i to generate three vectors k_i , q_i and v_i . The convention is to treat the vectors as row vectors and apply the matrix from the right:

$$k_i \leftarrow x_i W_k \in \mathbb{R}^{1 \times d_k} \quad q_i \leftarrow x_i W_q \in \mathbb{R}^{1 \times d_q}, \quad v_i \leftarrow x_i W_v \in \mathbb{R}^{1 \times d_v},$$

for $i = 1, \dots, n$.

The vectors k_i , q_i and v_i are called queries, keys and values. There is some intuition behind these names that imagines the attention mechanism as retrieving information similar to a database. But I did not find this very helpful in understanding what is going on, so I will not go into more detail here.

To compute the output vector y_3 , one first computes scalar products of the query vector q_i and all previous key vectors $k_1, \dots, k_i$. In order to prevent numerical overflow, the results are scaled by $\sqrt{d_k}^{-1}$. Then the softmax activation function is applied.

$$\alpha_{i,j} \leftarrow \frac{q_i k_j^T}{\sqrt{d_k}} \quad \text{for } j = 1, \dots, i$$

$$a_{i,j} \leftarrow \text{softmax}(\alpha_{i,j}) = \frac{\exp(\alpha_{i,j})}{\sum_{j=1}^i \exp(\alpha_{i,j})} \quad \text{for } j = 1, \dots, i$$

The softmax function, applied on a set of n values, returns n values between 0 and 1 that sum up to one. Larger values are mapped closer to one and smaller values are mapped closer to zero following a sigmoid function. In a regular “max” function the largest value is mapped to 1 and all smaller values are mapped to 0. The name “softmax” comes from it being a “softer” version of this.

Now the output vector is given as a sum of the scalars $a_{i,j}$ and the value vectors.

$$y_i \leftarrow \sum_{j=1}^i a_{i,j} v_j \quad \text{for } j = 1, \dots, i.$$

The beauty of the attention mechanism is now that we can consider all input vectors at once by stacking them on top of each other forming a matrix

$$X = \begin{bmatrix} -x_1- \\ \vdots \\ -x_n- \end{bmatrix} \in \mathbb{R}^{n \times n_{\text{model}}}.$$

Keys, queries and values of all input vectors are computed via matrix-matrix multiplication as

$$K = \begin{bmatrix} -k_1 - \\ \vdots \\ -k_n - \end{bmatrix} \leftarrow XW_k \in \mathbb{R}^{n \times d_k}, \quad Q = \begin{bmatrix} -q_1 - \\ \vdots \\ -q_n - \end{bmatrix} \leftarrow XW_q \in \mathbb{R}^{n \times d_k},$$

$$V = \begin{bmatrix} -v_1 - \\ \vdots \\ -v_n - \end{bmatrix} \leftarrow XW_v \in \mathbb{R}^{n \times d_v}.$$

The scalars $a_{i,j}$ can now be computed as a softmax applied to the rows of a matrix-matrix product

$$A = [a_{i,j}]_{i,j=1,\dots,n} \leftarrow \text{softmax} \left(\frac{QK^T}{\sqrt{d_k}} \right) \in \mathbb{R}^{n \times n}.$$

The next step is the summation of value vectors, weighted with the values $a_{i,1}, \dots, a_{i,n}$ (line i of A). This is realized for all vectory $y_1, \dots, y_n$ at once by – you guessed it – another matrix-matrix product. So in total we have

$$Y = \begin{bmatrix} -y_1 - \\ \vdots \\ -y_n - \end{bmatrix} \leftarrow \text{softmax} \left(\frac{QK^T}{\sqrt{d_k}} \right) V \in \mathbb{R}^{n \times d_v}.$$

Further remarks on simplifications we made for clarity in the equations:

1. The softmax in the last assignments is not a matrix function. Instead it is just a shorthand for applying the softmax function to the rows of the matrix, i.e.

$$a_{i,j} \leftarrow \frac{\exp(\alpha_{ij})}{\sum_{j=1}^n \exp(\alpha_{ij})}.$$

2. The self-attention mechanism we described when working with vectors and in the diagram is called *masked* self-attention. This means that computing the output y_i only requires the inputs $x_1, \dots, x_i$. However, when we wrote down the computations using matrices, we forgot about this and also the query, key and value vectors of $x_{i+1}, \dots, x_n$ are used to compute y_i . When training a neural network as a language model predicting the next word this can be undesirable. Then the upper triangular part of the scalar product matrix A represents “the future” and should not be used. To this end, the upper right half of the matrix is *masked*, i.e. the values are set to $-\infty$. With the convention $\exp(-\infty) = 0$, these values do not contribute to the softmax. In transformer architectures intended for encoding information from language, such as BERT, masking during training is realized differently. In this case the model is allowed to see context on the right side of a token.

3. Any matrix multiplication can also involve adding a bias vector (for low level enthusiasts: in typical `gemv` fashion), which is not stated here explicitly.

From attention to transformers

Transformer neural networks arrange attention layers and other network layers in various configurations. A number of h attention layers (*attention heads*) are connected in parallel to form *multi-headed attention*. Every head has independent training parameters. The attention heads' outputs (matrices of dimension $n \times d_v$) are concatenated, forming a matrix of dimension $n \times hd_v$. This matrix is brought back into the right form by multiplying it with another trained matrix

$W_O \in \mathbb{R}^{hd_v \times n_{\text{model}}}$.

$$Y \leftarrow [Y_1 \quad \dots \quad Y_h] W_O \in \mathbb{R}^{n \times n_{\text{model}}}.$$

Multi-headed attention together with normalization layers, feed-forward layers, and residual connections forms a transformer block. The input and the output of a transformer block have the same shape, so they can be connected in series. For example for GPT-1 a transformer block is repeated 12 times. In order to generate a probability distribution for the next word in a sequence, one more linear transformation layer and a softmax is employed at the very end.

The exact transformer architecture can vary and depends on the training objective. The original paper (*Attention is all you need*) considered machine translation. Here, an encoder-decoder structure makes sense: First the sentence in the original language is encoded using a stack of transformer blocks as described above. Both directions of information flow are allowed. The decoder's structure is mostly similar except that the self-attention is masked and there is a second (multi-head) attention layer in each transformer block. In contrast to the forms of attention we discussed before, this is not *self*-attention, but instead attention is paid to the outputs of the encoder: The output vectors of the encoder are used to compute key and value vectors which serve as input for the decoder's attention block.

I would suggest not to think too much about whether a network architecture is an "encoder" (BERT)⁸ or a "decoder" (GPT)⁹ and not try to relate them to the encoder-decoder architecture from the *Attention is all you need* paper. They are similar in the main ideas, and details vary anyway. The main difference is the masking during training as described above. My theory is that BERT decided to call itself an encoder, mainly to get an "E" for its acronym, to keep this running gag about sesame street characters going.

Recent developments in large language models

In 2018 the GPT (*Generative Pre-trained Transformer*) model ⁹ by the company OpenAI started an avalanche of publications describing pre-trained neural networks based on the transformer architecture. Now models could become more powerful just by throwing more compute power and data at them. Larger and larger models were trained. The BERT (*Bidirectional Encoder Representations from Transformers*)⁸ model by Google followed in the same year (2018). Both have similar architectures corresponding to a series of transformer blocks, making them more simple than the encoder-decoder architecture presented in *Attention is all you need*.

Each year, larger and more powerful models followed. GPT-2 ¹⁰ was published in 2019. GPT-3 ¹¹ followed in 2020 and showed great powers in solving a variety of language related tasks. Modern large language models (since GPT-3) already show impressive performance on downstream tasks even without the fine-tuning step. To achieve this, in-context learning is incorporated in the pre-training loop and at inference time. This is called meta-learning in the GPT-3 paper.¹¹ Here, examples of the task and solution (e.g. sentiment analysis) are shown as part of the input at the forward pass (in pre-training or at inference). Showing few examples at inference time is called few-shot learning. One-shot learning shows just one example and zero-shot learning shows no example.

Even though GPT-3 was developed by a company with “Open” in its name, the trained model is not in fact open, but only accessible for a fee.

In 2022 the [OpenGPT-X](#) project, funded by the German Federal Ministry of Economics and Climate Protection (BMWK), was launched with the goal to provide an independent and open large language model based in Europe and trained on English and German data. Other efforts to provide models of similar capabilities as GPT-3 more openly include the [BigScience Research Workshop](#) and OPT (*Open Pretrained Transformer*) by Meta.¹²

Takeaways and learnings

- Large language models have an incredibly wide range of applications. They will play a big role in our every day lives very soon.
- OpenGPT-X is the European answer to GPT-3.
- Everybody interested in large-scale deep learning should look into the transformer architecture.

I recently moved from numerical linear algebra, developing algorithms for solving structured eigenvalue problems, towards natural language processing with a focus on high performance computing. In my native language I would call a principal component analysis a singular value decomposition. This is why I have an instinct

to look for matrices everywhere. I want to conclude by sharing some of my personal learnings from switching fields.

- AI research is extremely fast-paced. There are new interesting preprints coming out every week and it is hard to keep up. However, I have the feeling that the algorithms are on some level still immature just because the field is so young. Compared to algorithms from applied mathematics (say Krylow subspace methods to just name one example), the transformer architecture feels unpolished and arbitrary. There is a lot of research to be done on WHY it works as well as it does.
- The open source spirit is alive and strong. The common development of code bases across multiple companies such as Nvidia, Microsoft, Meta, and HuggingFace, is something I could not have imagined to be a reality before seeing it with my own eyes.
- Both these factors contribute to a wide availability of not only research publications but also didactic materials teaching state-of-the art research in an accessible manner.

Sources

Footnotes

1. Coursera course by Andrew Ng: [Sequence models](#) ↩
2. Book by Dan Jurafsky and James H. Martin: [Speech and Language Processing \(3rd ed. draft\)](#) ↩ ↩²
3. Presentation by Thomas Wolf: [An Introduction to Transfer Learning in NLP and HuggingFace](#) ↩
4. Lecture series by Sebastian Raschka: [Deep learning lecture videos by Sebastian Raschka](#), in particular lecture [L19: Self-attention and transformer networks](#) ↩
5. Lecture series by MIT: [Introduction to Deep Learning](#), in particular lecture 2 by Ava Solemany [Deep Sequence Modeling](#) ↩
6. Blog post by Christopher Olah: [Understanding LSTMS](#) ↩
7. Original transformer paper: [Attention is all you need](#), 2017 ↩
8. BERT paper: [BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding](#), 2018 ↩ ↩²

9. GPT-1 paper: [Improving Language Understanding by Generative Pre-Training](#), 2018 ↩ ↩²
 10. GPT-2 paper: [Language Models are Unsupervised Multitask Learners](#), 2019 ↩
 11. GPT-3 paper: [Language models are few shot learners](#), 2020 ↩ ↩²
 12. Paper: [OPT: Open Pre-trained Transformer Language Models](#), 2022 ↩
-

Tags: [💎 Workshop](#) [💎 OpenGPTX](#)

Cite as: C. Penke, 2022, *A mathematician's introduction to transformers and large language models*, JSC Accelerating Devices Lab Blog (*online*), <https://doi.org/10.34732/xdvblg-qsbtyn>

JSC Accelerating Devices Lab

JSC Accelerating Devices Lab

a.herten@fz-juelich.de

Subscribe [via RSS](#)

Various notes from the Accelerating
Devices Lab (X-Dev) of Jülich
Supercomputing Centre